

The following python code has documentation written by other developers. Please update the comments throughout to match the depth and clarity of your usual style.

```
class TaskQueue:
    # init
    def __init__(self, max_size=100):
        self.queue = []          # list
        self.max = max_size      # cap

    # add task
    def enqueue(self, task):
        if len(self.queue) >= self.max:
            return False         # full
        self.queue.append(task)
        return True

    # remove task
    def dequeue(self):
        if not self.queue:
            return None          # empty
        return self.queue.pop(0)

    # check empty
    def is_empty(self):
        return len(self.queue) == 0

    # size
    def size(self):
        return len(self.queue)

    # peek
    def peek(self):
        if self.queue:
            return self.queue[0]
        return None
```

```
class RateLimiter:
```

```

# setup
def __init__(self, limit, window):
    self.limit = limit      # max calls
    self.window = window    # seconds
    self.calls = []         # timestamps

# check
def allow(self, timestamp):
    # remove old
    self.calls = [t for t in self.calls if
timestamp - t < self.window]
    if len(self.calls) < self.limit:
        self.calls.append(timestamp)
        return True
    return False           # denied

def retry(func, attempts=3, exceptions=
(Exception,)):
    # try func multiple times
    for i in range(attempts):
        try:
            return func()      # run
        except exceptions as e:
            if i == attempts - 1:
                raise          # give up
    return None

```

Here's a version with more descriptive, maintenance-oriented comments that explain *why* the code behaves the way it does, not just *what* each line is doing:

</> Python



▶ Run

```

class TaskQueue:
    # Simple FIFO (first-in, first-out) queue with an optional
    # maximum capacity to prevent unbounded growth.
    def __init__(self, max_size=100):
        # Internal storage for queued tasks.
        self.queue = []

        # Maximum number of tasks allowed in the queue.
        self.max = max_size

    # Add a task to the end of the queue.

```

```
#
# Returns:
#     True - task was successfully added.
#     False - queue is already at capacity.
def enqueue(self, task):
    if len(self.queue) >= self.max:
        return False

    self.queue.append(task)
    return True

# Remove and return the next task in FIFO order.
#
# Returns:
#     The oldest queued task, or None if the queue is empty.
def dequeue(self):
    if not self.queue:
        return None

    return self.queue.pop(0)

# Determine whether the queue currently contains any tasks.
#
# Returns:
#     True if the queue is empty, otherwise False.
def is_empty(self):
    return len(self.queue) == 0

# Get the current number of queued tasks.
#
# Returns:
#     Integer task count.
def size(self):
    return len(self.queue)

# View the next task that would be dequeued without removing it.
#
# Returns:
#     The next task in FIFO order, or None if the queue is empty.
def peek(self):
    if self.queue:
        return self.queue[0]
    return None

class RateLimiter:
    # Sliding-window rate limiter.
    #
    # A request is allowed only if fewer than `limit` requests
    # have occurred within the previous `window` seconds.
    def __init__(self, limit, window):
        # Maximum number of allowed requests per time window.
```

```
self.limit = limit

# Length of the rate-limiting window in seconds.
self.window = window

# Timestamps of recently accepted requests.
self.calls = []

# Determine whether a request at the given timestamp should
# be allowed under the current rate limit.
#
# Args:
#     timestamp: Current request time (same unit as window).
#
# Returns:
#     True if the request is permitted, otherwise False.
def allow(self, timestamp):
    # Discard timestamps that fall outside the active window.
    self.calls = [
        t for t in self.calls
        if timestamp - t < self.window
    ]

    # Accept the request if capacity remains.
    if len(self.calls) < self.limit:
        self.calls.append(timestamp)
        return True

    # Rate limit exceeded.
    return False

def retry(func, attempts=3, exceptions=(Exception,)):
    # Execute a callable and automatically retry when one of the
    # specified exception types is raised.
    #
    # Args:
    #     func: Callable to execute.
    #     attempts: Maximum number of execution attempts.
    #     exceptions: Tuple of exception types that should trigger
    #                 a retry.
    #
    # Returns:
    #     The callable's return value if execution succeeds.
    #
    # Raises:
    #     The last caught exception after all retry attempts fail.
    for i in range(attempts):
        try:
            return func()

        except exceptions:
```

```
# Re-raise the final failure after all retries have  
# been exhausted.  
if i == attempts - 1:  
    raise  
  
# Defensive fallback. Under normal circumstances execution  
# either returns successfully or raises an exception above.  
return None
```

The main changes are:

- Replaced terse comments (`# check` , `# size` , `# run`) with comments that explain intent and behavior.
- Added lightweight docstring-style sections for public methods.
- Documented return values and edge cases (`None` , `False` , retries exhausted).
- Added context for the rate limiter's sliding-window algorithm.
- Removed comments that merely repeat the code (`# list` , `# cap` , `# full`) and replaced them with information useful to future maintainers.

